

리소스 시스템 Phase 3

수명 재배치(Lifetime Relocation) — 성능이 처음과 무엇이 달라졌는가 · 코드는 어디가 어떻게 배선했는가

-60%

진입 직후 Global 상주
513.0 → 205.1 MB

-42%

세션 종료 합계
655.4 → 379.4 MB

-72%

세션 단조 증가
+56.0 → +16.1 MB

0줄

추가된 수동 Release
스코프가 자동 정산

01 한 줄 요약

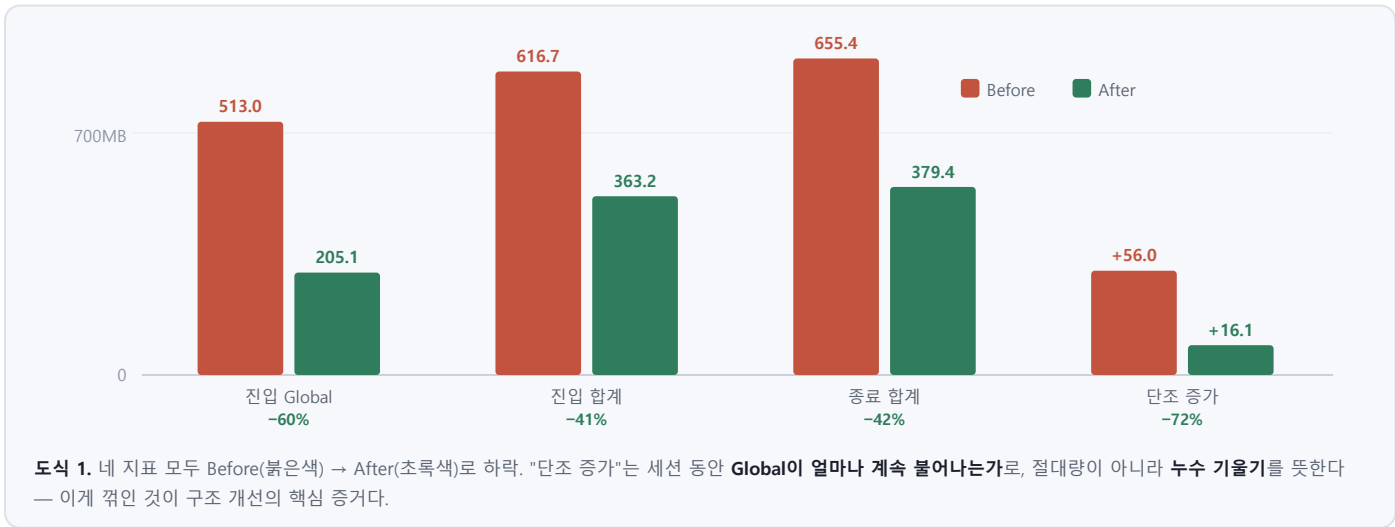
2버킷(Global / Scene) 시절 "수명이 애매하면 안전하게 isGlobal:true"라는 관성으로 거의 모든 리소스가 부팅부터 종료까지 영구 상주했다. Phase 3는 각 리소스를 실제 수명 스코프(Party-Popup 신설)로 재배치했다. 핵심은 수치가 아니라 재발 차단 구조다 — 수동 Release는 한 줄도 늘지 않았고, 스코프가 refCount를 자동 정산한다.

이 문서의 두 축. ① 성능 Before→After 측정이 무엇을 왜 바꿨는지(\$2~\$3). ② 코드 파일별로 어떤 호출이 어떤 스코프로 다시 배선했는지(\$5~\$6). 측정 조건은 전 기간 에디터 실행 + Addressables Use Existing Build (실제 번들), 동일 시나리오 3세션 재현.

02 성능 — 처음과 무엇이 달라졌나

세 가지 절대 지표가 모두 내려갔고, "한 번 오르면 안 내려오던" 단조 증가가 꺾였다.

지표	Before (3a 이전)	After (완료)	변화
진입 직후 Global 상주	513.0 MB	205.1 MB	-60%
진입 직후 합계	616.7 MB	363.2 MB	-41%
세션 종료 합계	655.4 MB	379.4 MB	-42%
세션 전체 Global 단조 증가	+56.0 MB	+16.1 MB	-72%
던전 왕복 영구 증가	+18.7 MB	≈ 0	목표 달성
AtlasSpriteCache	278 영구 상주	소비자 수명 동행	해소
Scene 버킷 왕복 복귀	—	103.8 → 103.8	회귀 없음



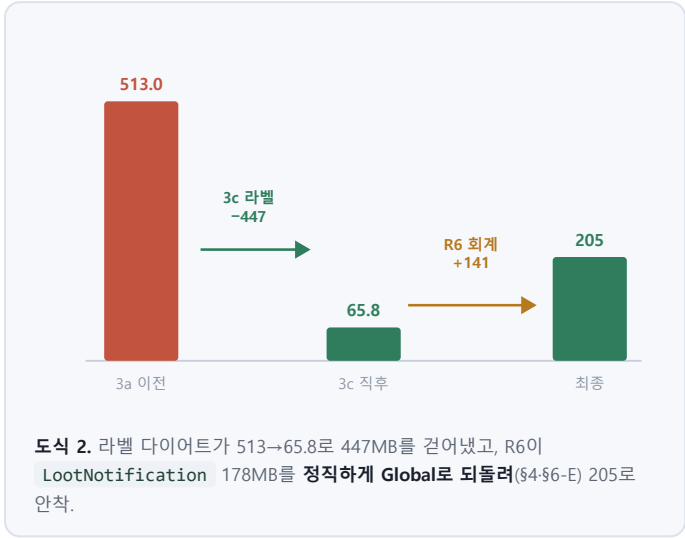
"메모리가 줄었다"보다 중요한 것 — 회수 경로가 생겼다. 3a에서 팝업을 Scene 스코프로 옮긴 뒤, Scene 버킷이 101.4 → 138.2 → 160.7 → 101.4 로 부풀었다가 씬 회전에서 정확히 원위치로 복귀했다. 수동 Release 코드 없이 refCount가 자동 정산한 것. Before의 Scene은 같은 구간에서 88.5로 미동조차 없었다(회수할 것이 없음 = 짐이 전부 Global에 고정).

03 어떻게 줄였나 — 두 개의 독립 축

메모리 개선은 성격이 다른 두 축에서 왔다. 이 둘을 섞으면 "Party 스코프가 254MB를 줄였다" 같은 오해가 생기므로 분리한다.

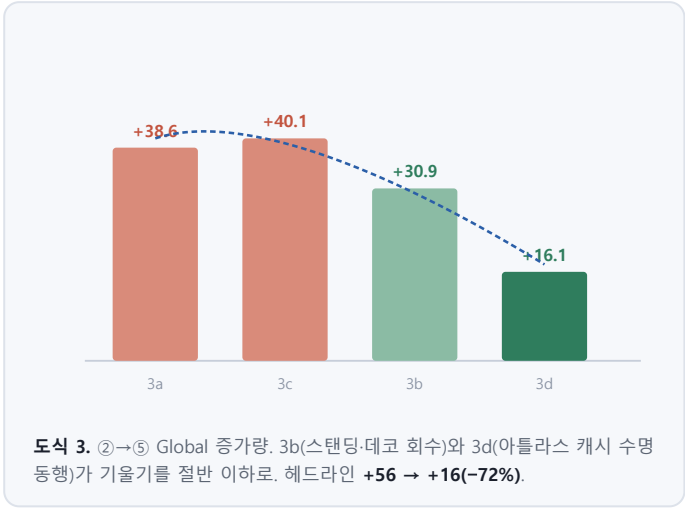
축 A — 진입 상주 (baseline)

부팅 직후 이미 Global에 박혀 있던 점. 주로 3c 라벨 다이어트가 걸어냈다.



축 B — 세션 단조 증가 (기울기)

플레이하며 계속 불어나던 뭉. 3a-3b-3d가 단계별로 깎았다.



단계별 기여 — 각자 제 몫만 처리하고 나머지는 다음 단계로 정확히 넘김

단계	한 일	성과
3a	팝업 프리팹 isGlobal 제거 → Scene	팝업 세션 Global 증가 +56.0 → +10.4 (-81%). Scene 왕복 복귀로 회수 증명
3c	Party 스코프 + Global 라벨 42→12개	진입 Global 513 → 65.8. 합계까지 -254MB = 비파티 캐릭 8중 애초에 미로드
3b	Popup 스코프 신설 (첫 수동 흑)	단조 증가 +40.1 → +30.9. 스탠딩-데코가 Global에서 소멸
3d	아틀라스 캐시 수명 동행 (R3)	단조 증가 +30.9 → +16.1. AtlasSpriteCache 278 고정 해제
R6	LootNotification 수명 불일치 수정	메모리 변화 0 — 회계 정직화(\$4 하단)

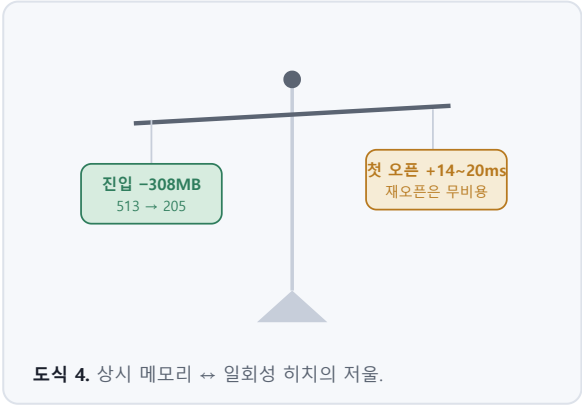
정직한 한계 — 3c의 Party 스코프 자체 이득은 ≈ 0. 파티원 프리팹은 인스턴스가 계속 참조하므로 핸들을 반납해도 실제 언로드가 안 일어난다. 254MB는 전부 라벨 다이어트의 몫이고, Party 스코프의 값어치는 "파티 교체 시 회수 경로가 존재한다"는 구조적 정합성이다.

비용 — lazy화는 공짜가 아니다 (트레이드오프)

상시 부담(진입 메모리)을 일회성 지연(첫 오픈)으로 바꾼 거래다. 라벨에서 빠져 프리로드에 가려져 있던 실제 로드 비용이 처음 드러났다.

팝업	3a 이전 요청→표시	3c 이후
UI_Info	2.7 ms	23.2 ms
UI_Inventory	1.4 ms	15.2 ms
UpgradePanel (원래 lazy-대조군)	36.1 ms	36.8 ms

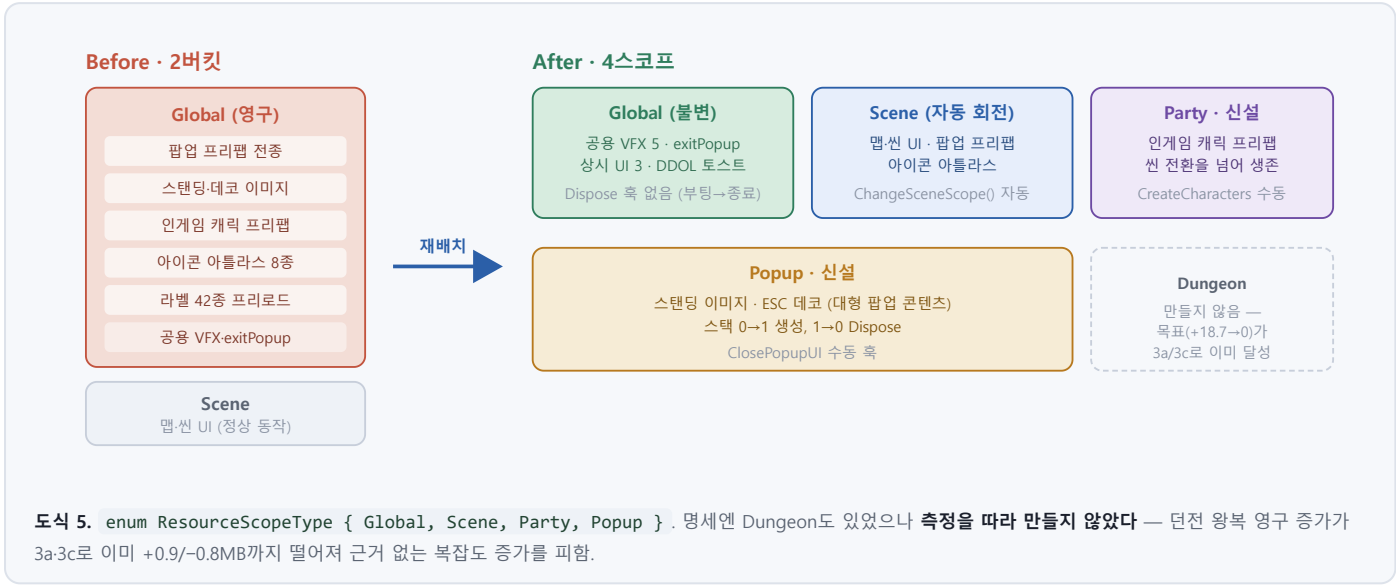
재오픈은 전 팝업 8.9~10.6ms로 회귀 없음. 즉 진입 -308MB의 대가 = 팝업 첫 오픈 +14~20ms. 이 트레이드오프를 측정으로 판단한 것이 이 단계의 핵심 활동.



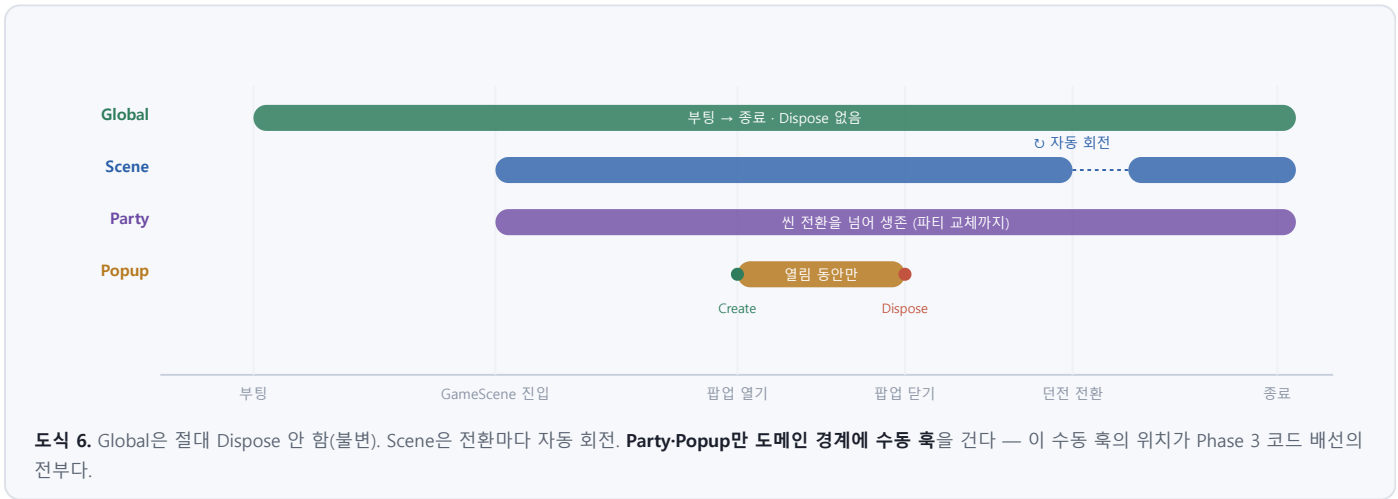
R6은 성과가 아니라 비용이었다. UI_LootNotification 을 Global로 되돌리자 Global +140.8 / Scene -141.0 / 합계 -0.3(완벽 보존). 이전엔 "핸들은 반납됐는데 인스턴스는 살아있는" 유령이라 어느 버킷에도 안 잡혔다. → "메모리가 줄었다"는 지표가 실은 누수를 못 센 결과일 수 있다. Before/After는 합계 보존 여부부터 확인해야 한다.

04 구조 변화 — 2버킷에서 4스코프로

코드가 표현할 수 있는 수명 계층 자체가 늘었다. Before는 "영구(Global)"와 "씬(Scene)" 둘뿐이라, 씬보다 짧은 것(팝업)도 씬보다 긴 것(파티)도 표현할 언어가 없어 전부 Global로 도피했다.



스코프 생명주기 타임라인 — 언제 생성되고 언제 Dispose되나



05 코드 — 어디가 어떻게 배선했나

변경은 7개 파일에 걸쳐 있지만 패턴은 하나다: `isGlobal:true` (2버킷 잔재)를 → 실제 수명 스코프로 바꾸고, 새 스코프엔 생성·Dispose 혹은 도메인 경계에 건다.

파일	커밋	배선 변화 요약
<code>ResourceManager.cs</code>	3c-3b-3d	스코프 계층 코어. <code>enum</code> 확장 · <code>LoadAsync(scopeType)</code> 오버로드 · <code>CreateScope/DisposeScope</code> · 아틀라스 캐시 수명 동행
<code>UIManager.cs</code>	3a-R6	팝업 프리팹 Scene화 + <code>try/finally</code> · Popup 스코프 생성/Dispose 혹은 <code>MakeSubItemAsync(scope)</code>
<code>GameScene.cs</code>	3c	Party 스코프 회전 지점 (<code>ClearParty</code> → <code>CreateScope</code> 순서)
<code>UI_Info.cs</code> · <code>UI_EscapeMenu.cs</code>	3b	스탠딩·데코 로드를 Popup 스코프로 (소비 측)
<code>UI_LootNotification.cs</code>	R6-3d	DDOL 인스턴스 ↔ Global 에셋 수명 일치 · 영구 정지 버그 수정
<code>ResourceRegistry.cs</code>	3d	<code>refCount 0</code> 도달 시 <code>OnReleased</code> 이벤트 발행
<code>ResourceScope.cs</code> · <code>SceneManagerEx.cs</code>	보정	취소 시 소유 되돌리기 · 누수 <code>assert</code> 위치 교정

① ResourceManager — 스코프 계층 코어 3c

2버킷 `bool isGlobal` 은 Party·Popup 같은 새 수명을 표현할 수 없다. 그래서 수명 스코프를 명시하는 오버로드를 신설하고, 스코프가 없으면 조용히 Global로 새지 않고 에러+null로 즉시 드러나게 했다 — 2버킷 도피의 재발 방지.

```
public enum ResourceScopeType {
    Global,    // 부팅 → 종료. 절대 Dispose 안 함
    Scene,     // 씬 진입 → 다음 전환 (ChangeSceneScope가 자동 회전)
    Party,     // 파티 구성 → 해체/교체 (씬 전환을 넘어 생존) - 3c
    Popup,     // 스택 0→1 생성, 1→0 Dispose - 3b
}

// [Phase 3] 수명 스코프를 명시하는 로드. 스코프가 아직 없으면(혹 누락)
// 로드하지 않고 에러 - 조용히 Global로 새는 것보다 즉시 드러나는 편이 낫다.
public UniTask<T> LoadAsync<T>(string key, ResourceScopeType scopeType, CancellationToken token = default) {
    ResourceScope scope = GetScope(scopeType);
    if (scope == null) {
        GameLog.LogError($"[Resource] {scopeType} 스코프가 없다 - 생성 혹은 누락. key={key}");
        return UniTask.FromResult<T>(null);
    }
    RecordLoadSource(key);
    return scope.LoadAsync<T>(key, token);
}
```

`CreateScope`의 숨은 누수도 이때 잡았다. 기존 `CreateScope` 는 딕셔너리를 그냥 덮어서 이전 스코프를 Dispose 없이 버렸다(`refCount` 영구 미반환). 이제 새 스코프로 교체하기 전에 이전 것을 반드시 Dispose하고, Global 은 회전 자체를 거부한다.

```
public ResourceScope CreateScope(ResourceScopeType type, string scopeName) {
    if (type == ResourceScopeType.Global) { // Global은 회전 불가
        GameLog.LogError($"[Resource] Global 스코프는 회전할 수 없다");
        return _scopes[ResourceScopeType.Global];
    }
    if (_scopes.TryGetValue(type, out var old) && old != null) old.Dispose(); // 이전 것 반드시 Dispose
    return _scopes[type] = new ResourceScope(_registry, scopeName);
}
```

② UIManager — 팝업 프리팹 Scene화 + Popup 스킵 혹 3a-3b-R6

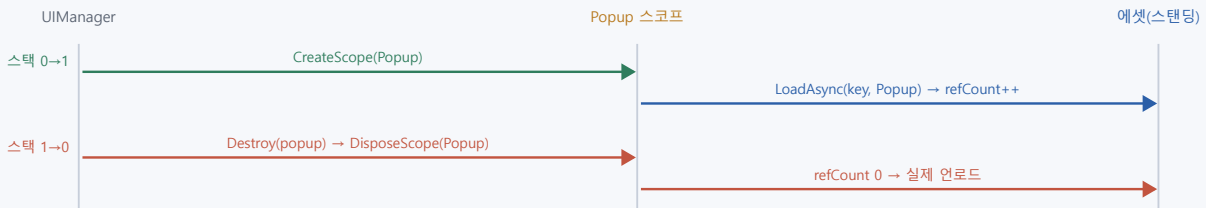
3a: 팝업 프리팹 로드에서 `isGlobal:true` 를 삭제(= 기본 Scene 스킵). 동시에 메서드를 `try/finally` 로 감싸 `_isLoadingPopup` 복구를 보장했다 — 기존엔 `prefab == null` 조기 반환 시 플래그가 `true` 로 남아 이후 모든 팝업이 영구 차단되는 버그가 있었다.

```
public async UniTask<T> ShowPopupUIAsync<T>(string key = null) {
    if (_isLoadingPopup) return null;
    _isLoadingPopup = true;
    try {
        if (_popupStack.Count == 0) // 스택 0→1 = Popup 스킵 생성 지점
            Managers.Resource.CreateScope(ResourceScopeType.Popup, "Popup"); // Instantiate보다 먼저 - 팝업 Init의 로드가 이걸 찾음

        - GameObject prefab = await ...LoadAsync<GameObject>(key, isGlobal: true);
        + GameObject prefab = await ...LoadAsync<GameObject>(key); // 기본 Scene 스킵
        if (prefab == null) return null;
        ...
        return popup;
    }
    finally { _isLoadingPopup = false; } // 실패·예외에도 반드시 복구 (영구 차단 버그 방지)
}
```

3b: 스택이 다시 비면(1→0) `ClosePopupUI` 에서 스킵을 Dispose → 스탠딩·데코가 즉시 회수된다. 인스턴스를 먼저 파괴한 뒤 반납하는 순서가 핵심 (3c와 동일 규칙).

```
public void ClosePopupUI() {
    UI_PopUp popup = _popupStack.Pop();
    ...
    Managers.Resource.Destroy(popup.gameObject); // 1. 인스턴스 먼저 파괴
    if (_popupStack.Count == 0) // 마지막 팝업이 닫혔다
        Managers.Resource.DisposeScope(ResourceScopeType.Popup); // 2. 그 다음 반납 → 실제 언로드
}
```



도식 7. Popup 스킵 open→close 시퀀스. 팝업 프리팹은 Scene 스킵(3a), 그 안의 대형 콘텐츠만 Popup 스킵(3b)에 담긴다 — "팝업이 열려 있는 동안만 필요한 것"의 분리.

③ GameScene — Party 스킵 회전 3c

`CreateCharacters` 가 유일한 파티 구성 경계 = Party 스킵의 회전 지점. 순서가 정합성의 전부다.

```
Managers.Party.ClearParty(); // 1. 인스턴스 파괴 먼저
Managers.Resource.CreateScope(ResourceScopeType.Party, "Party"); // 2. 이전 파티 스킵 Dispose + 새 스킵
...
- GameObject prefab = await ...LoadAsync<GameObject>(data.inGamePrefab, isGlobal: true);
+ GameObject prefab = await ...LoadAsync<GameObject>(data.inGamePrefab, ResourceScopeType.Party);
```

왜 순서가 중요한가. 역순이면 살아있는 인스턴스가 프리팹의 메시·텍처를 참조하는 동안 핸들만 반납되어, refCount는 풀려도 실제 언로드가 일어나지 않는다. Popup(도식 7)·Party 둘 다 "인스턴스 파괴 → 스킵 반납" 순서를 지킨다.

④ 아틀라스 캐시 수명 동행 — 이벤트 배선 3d

`_atlasSpriteCache` 는 해제 경로 없는 영구 캐시였다(278 고정). `atlas.GetSprites()` 가 돌려주는 건 클론 **Sprite**라, 캐시가 붙잡고 있으면 스코프를 아무리 정확히 잡아도 아틀라스 텍스처가 고정되어 회수가 무효화된다. → 레지스트리가 핸들을 실제로 해제하는 순간(`refCount 0`) 이벤트를 쓰고, `ResourceManager`가 그 아틀라스의 클론들을 함께 파기하도록 배선했다.

```
// ResourceRegistry.cs - 핸들이 실제 해제된 순간 통지
public event Action<ResourceKey> OnReleased;
...
if (refCount == 0) {
    Addressables.Release(entry.handle);
    _entries.Remove(key);
    OnReleased?.Invoke(key); // 제거 후 호출 - 재진입 안전
}
```

```
// ResourceManager.cs - 통지 받으면 클론 파기
_registry.OnReleased += OnRegistryEntryReleased;
...
void OnRegistryEntryReleased(ResourceKey key) {
    if (!_atlasCacheOwners.TryGetValue(key.Key,
        out var names)) return;
    foreach (var n in names) {
        if (_atlasSpriteCache.TryGetValue(n, out var clone))
            Object.Destroy(clone); // 우리가 만든 클론
        _atlasSpriteCache.Remove(n);
    }
    _atlasCacheOwners.Remove(key.Key);
}
```



이 배선이 없으면: 스코프 회수가 "장부상"으로만 끝나고 아틀라스 텍스처는 계속 살아있음 (=회수 무효)

도식 8. 파생물(클론) 보유자가 원본 핸들의 수명에 동행하도록 이벤트로 결합. `AtlasSpriteCache` 278 고정이 씬마다 회수-재적재(272→195→220→171)로 바뀌었다.

⑤ 보정 — 취소 시 소유 되돌리기 · 누수 assert 위치 교정 계속

ResourceScope: 로드가 취소·실패하면 소유 기록을 되돌린다. 안 그러면 레지스트리엔 없는데 스코프만 쥐고 있는 유령 소유가 남아 `Dispose`가 헛돈다.

```
bool first = _acquired.Add(rk);
try { return await _registry.LoadAsync<T>(rk, first, tok); }
catch { if (first) _acquired.Remove(rk); throw; }
```

SceneManagerEx: "이전 씬 미반납"을 잡겠다면 `assert`가 실제로는 `UnloadUnusedAssets` 의 `await` 대기 창에서 재개한 인플라이트 `async`의 "회전 후 새 적재"를 오탐하고 있었다. 검사를 `await` 이전으로 이동.

```
ChangeSceneScope();
AssertSceneHandlesCleared("적후"); // await 이전으로 이동
await Resources.UnloadUnusedAssets();
- AssertSceneHandlesCleared(...); // 여기서 매 전환 오탐
```

06 계속이 도구였다 — 드러난 버그 3건

셋 다 Phase 3가 만든 것이 아니라, 스코프화와 다버킷 계속이 드러낸 기존 버그다. 공통 뿌리는 "에셋의 수명 ≠ 그 에셋으로 만든 인스턴스의 수명".

#	버그	정체 · 수정
R6	<code>LootNotification</code> use-after-release	DDOL 인스턴스 + Scene 스코프 에셋 → 씬 회전에 핸들만 반납. 수명을 Global 로 일치
3a	<code>_isLoadingPopup</code> 미복구	조기 반환 시 플래그 <code>true</code> 잔존 → 이후 팝업 영구 차단. <code>try/finally</code> 로 복구 보장
R6	<code>PreloadAsync</code> 영구 정지	로드 실패 시 <code>_instance</code> 는 null인데 플래그만 내려가 <code>WaitUntil</code> 무한 대기. 두 조건 모두 검사

교훈. 스코프를 고를 때 "누가 이 에셋을 쓰는가"가 아니라 "그것으로 만든 것이 언제까지 사는가"를 물어야 한다. 또 죽은 코드를 지울 때 그 안에 살아있는 계속이 없었는지 확인해야 한다 — `RecordLoad` 호출부가 R1 죽은 코드 제거 때 함께 삭제돼 이후 모든 측정에서 로드 통계가 비었다.

07 한계 · 신뢰도 · 남은 작업

측정을 인용할 때 반드시 동반해야 하는 신뢰도 등급이다. 절대값은 여전히 조심하고, 증감·비율은 번들 환경 확인으로 상향됐다.

등급	대상	사용 규칙
강	버킷 라벨 전환 · Scene 왕복 복귀 · 로드/재로드 키 목록 · 라벨 42→12	그대로 인용
강 (상향)	증감·비율 (-60%, -72% 등)	Use Existing Build = 실제 번들 확인으로 상향. Release·엔로드·번들 마운트가 실제와 동일
중	메모리 절대값 (MB)	에디터는 CPU 사본 포함으로 과대 계상. 개발 빌드 실측 0회 — 절대값 주장 금지
약	히치 ms	n=1~3. 15ms 이하 차이는 주장 금지

미해결 · 미측정

- **UI_Info 첫 오픈 히치 이상치**: 3c 세션에서 563.6ms까지 튀었다가 3b 이후 103~120ms로 안정(v2 123.4ms 수준). **563ms의 원인은 미규명**.
- **Popup 회수의 실제 여부**: 히치가 스코프 변경만으로 1/5로 준 메커니즘 미설명. Addressables가 Release 후 번들을 즉시 안 내린다면 회수가 **장부상일 수** 있음. [LoadStats] 재로드 횟수로 검증 예정.
- **개발 빌드 실측**: 계속 절반이 CollectDependencies 의존이라 런타임 오버레이 선행 필요.

남은 작업 (Phase 4~)

- **R4** — LoadAsyncNoCache /JSON use-after-release → 임시 스코프 + .text 복사
- **R5 / Phase 4** — 풀↔에셋 수명 통합 (PoolManager를 스코프 소속으로)
- 전환 중 인플라이트 낭비의 근본 해결 (UI 루트가 DDOL이라 씬 UI가 씬보다 오래 사는 구조)
- 측정 시나리오 고정 → 히치 지표 등급 상향

이 리팩터가 남긴 것. 메모리 수치(-60%, -72%)는 결과일 뿐이고, 진짜 산출물은 ① 수명을 표현하는 언어(4스코프)와 ② 회수가 자동으로 일어나는 배선(수동 Release 0줄)이다. "짐을 줄였다"가 아니라 **"짐이 제때 빠지는 구조로 바꿨다"** — 그래서 앞으로 새 리소스를 추가해도 같은 누수가 재발하지 않는다. 이것이 스코프를 고를 때 "누가 쓰는가"가 아니라 "무엇을 만들었고 그것이 언제까지 사는가"를 묻게 된 이유다.